

1) Summary

This project is a **customer support chatbot** that answers questions from your existing FAQs and knows when to hand off to a human. It uses a technique called **Retrieval-Augmented Generation (RAG)**: before the bot writes anything, it first **finds the most relevant FAQ entries**, then answers using only that information. Answers include **inline citations** like `[FAQ-12]` so users can see where facts came from.

If the match is weak, the bot **doesn't guess**. It asks one short clarifying question. If the user says “create ticket” (or if confidence stays low), the bot **summarizes the whole chat**—issue, steps tried, likely environment (web/iOS/Android/API), and FAQ references—and **opens a support ticket**.

Everything is built on **student-friendly tools**:

- **Next.js + Vercel** for the UI and API routes (easy to deploy, free tier friendly)
- **Supabase** for database, authentication, and vector search via **pgvector**
- **Small, fast models**: BGE for embeddings and **Groq Llama-3.1-8b-instant** for writing answers & summaries

It's **production-ready** in the basics (auth, RLS, error handling, guardrails) but intentionally **simple to operate**, so you can extend it later (more data sources, admin tools, analytics, Slack/Jira hooks).

2) Problem Statement

The pain today

- Users ask the same questions again and again (“reset password?”, “invoice download?”, “change plan?”). Human agents spend time repeating answers that already exist in FAQs.
- “General” chatbots often **hallucinate** or give **inconsistent** answers because they aren't grounded in your company's content.

- When automation fails, the hand-off is clumsy: tickets lack a clear summary, steps tried, or context—so agents must **start from scratch**.

What we want instead

- A bot that **answers accurately** from **your** FAQs and shows **receipts** (citations).
- A bot that **knows its limits**: if it's not confident, it **asks a focused follow-up** or **escalates cleanly**.
- When escalating, the bot should **do the busywork**: create a **well-formed ticket** with the problem, context, steps tried, environment, and any relevant FAQ links—so agents can help faster.

Constraints

- Must be **cheap** (student/early-stage budgets).
- Must be **easy to deploy** (no heavy infra).
- Must be **safe** (auth, per-user data isolation, no “random” answers).

Success looks like

- Users get **faster answers** with **fewer escalations**.
- Agents receive **better tickets** (clear summaries), cutting resolution time.
- Costs stay low enough to run continuously during development and demos.

3) Objectives & Scope

Primary objectives

1. **Trustworthy answers from FAQs**
Use RAG to retrieve the best matches and answer **only** from those passages. Add **[FAQ-id]** citations for transparency.

2. **Guardrails to prevent guessing**

Set **similarity thresholds**. Below threshold → ask **one** clarifying question instead of fabricating.

3. **One-click ticket creation from chat**

Summarize the entire conversation (title, problem, steps tried, environment, FAQ refs). Write a clear, human-ready ticket.

4. **Accounts & history**

Let signed-in users see previous conversations and tickets in a **left sidebar**; open the latest chat by default.

5. **Student-friendly ops**

Ship on Vercel + Supabase; keep models small and fast; keep code modular so components can be swapped later.

Measurable outcomes (how we know it works)

- **≥ 80%** of “common” questions answered without escalation (after FAQs are seeded well).
- **0 hallucination complaints** in normal use (guardrails kick in on low similarity).
- Tickets contain **title, steps, environment, and FAQ refs** consistently.
- Median answer time **< 1.5s** for retrieval; **< 3s** for LLM answers (typical loads).

Out of scope for v1 (by design)

- Multi-channel integrations (Slack/Zendesk/Jira/Linear).
- Ingesting large doc corpora beyond FAQs.
- Advanced analytics/dashboards (we keep logging simple first).

Assumptions

- FAQs are reasonably complete and up-to-date.
- Support team checks tickets created by the bot and closes the loop.

4) Users & Scenarios

Personas

- **Customer (end-user):** wants quick, reliable answers. If not satisfied, wants an **easy escalation** path that doesn't repeat everything to a human.
- **Support Agent:** wants **clean tickets** with the right context and fewer back-and-forth messages.
- **Admin/Owner:** wants something that **deploys in minutes**, runs cheaply, and is straightforward to maintain.

Core journeys

1. Quick Answer (Happy Path)

- User: "How do I reset my password?"
- Bot: Retrieves top FAQ, answers in **1–3 short paragraphs**, cites [\[FAQ-12\]](#).
- Outcome: No human needed.

2. Low Confidence → Follow-Up

- User: "I'm stuck on login; it keeps looping."
- Bot: Retrieval is borderline; **asks one focused question** ("Do you see an error or a blank screen?") and suggests related FAQs.
- Outcome: Often resolved with a single extra detail.

3. Escalation → Ticket

- User: "create ticket."
- Bot: Loads **entire chat**, removes "create ticket" noise, infers environment (web/iOS/Android/API), compiles **title + summary + steps + FAQ refs**, creates ticket, confirms "Ticket #123 created."

- Agent: Sees a **ready-to-work** ticket instead of a blank form.

State & privacy

- **Anonymous users** can chat; creating a ticket triggers **login** first (the bot preserves a draft summary to finish after login).
- **Signed-in users** see their **chat history** and **tickets**; RLS in Supabase ensures users only see **their own** data.

Accessibility & UX

- Two-pane layout (sidebar + thread) for clarity.
- Citations are visible and clickable.
- Minimal prompting to the user; clear error messages if something fails.

5) Architecture (High-Level)

What happens when a user asks a question?

1. The browser shows a **two-pane UI** (left: chat history, right: chat thread).
2. The message is sent to a **serverless API route** in Next.js (running on Vercel).
3. The API:
 - turns the question into an **embedding** (a numeric vector) using a small model,
 - searches Supabase/Postgres with **pgvector** to find the **closest FAQ entries**,
 - decides whether confidence is high enough to answer,
 - either writes a **clean, cited answer** (using a small LLM) or asks **one short follow-up**.
4. If the user says “**create ticket**”, the API loads the **entire conversation**, summarizes it into a **clear ticket**, and saves it in Supabase.

Why this shape works

- The browser stays simple (no secrets, no heavy logic).
- The API routes do the “smart” parts and keep your keys safe.
- Postgres holds both your **data** (FAQs, chats, tickets) and **vectors** (for fast similarity search), which keeps ops simple.

Sequence at a glance (Mermaid diagram)

sequenceDiagram

participant U as User (Browser)

participant A as Next.js UI

participant API as Next.js API Routes

participant DB as Supabase (Postgres+pgvector)

participant HF as HF Inference (Embeddings)

participant G as Groq (LLM)

U->>A: Type question

A->>API: POST /api/answer or /api/chat

API->>HF: Embed question (BGE)

HF-->>API: 384-d vector

API->>DB: RPC: match_faq(vector, top_k, min_sim)

DB-->>API: Top matches + similarity

alt confident

API->>G: Draft concise answer w/ citations

G-->>API: Answer text

API-->>A: JSON { answer, sources }

else not confident

API->>G: Ask 1 follow-up

G-->>API: Short question

API-->>A: JSON { clarify, suggestions }

end

opt user says "create ticket"

API->>DB: Load full conversation

API->>G: Summarize conversation → JSON (title, steps, env, refs)

G-->>API: Ticket draft

API->>DB: Insert ticket

API-->>A: JSON { ticket_created }

end

Security & privacy touchpoints

- **Auth cookies** are read server-side using Supabase helpers; no secrets in the browser.
 - **Row-Level Security (RLS)** ensures users can only read/write their own conversations and tickets.
 - **Service role key** is used only where needed (e.g., a protected seeding route), never sent to the browser.
-

6) Data Model (Core Tables)

FAQs & embeddings

- `faq (id, question, answer, created_at)`
- `faq_embeddings (faq_id, embedding vector(384))`
 - `embedding` uses `pgvector` (cosine/inner-product similarity)

Chat

- `conversations (id, user_id, created_at)`
- `messages (id, conversation_id, role ['user'|'assistant'], content, created_at)`

Tickets

- `tickets (id, user_id, conversation_id, title, summary, status, priority, created_at)`

7) Technologies Used

Frontend & Hosting

- **Next.js (App Router)** – pages + API routes in one codebase
- **Tailwind CSS** – clean, accessible UI with minimal custom CSS
- **Vercel** – serverless deploys, good free tier

Backend & Database

- **Supabase** – Postgres, Auth, Storage, built-in **RLS**
- **pgvector** – stores embeddings and does fast similarity search
- **SQL RPC** – small Postgres functions for retrieval (keeps logic close to data)

LLM & Embeddings

- **Hugging Face Inference API** – [BAAI/bge-small-en-v1.5](#) for 384-dim embeddings
- **Groq** – [llama-3.1-8b-instant](#) for quick, affordable answer writing and ticket summaries

Why this stack

- **Fast to ship:** Next.js + Vercel deploys in minutes.
- **Low cost:** small models + serverless = cheap to run.
- **Simple mental model:** one database stores **everything** (data + vectors).
- **Modular:** you can swap models or add new data sources without a rewrite.

Key environment variables

- `NEXT_PUBLIC_SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_ANON_KEY`
 - `SUPABASE_SERVICE_ROLE_KEY` (server-only; use sparingly)
 - `HF_TOKEN` (Hugging Face)
 - `GROQ_API_KEY` (Groq)
-

8) Models Used (and Why)

Embeddings: BAAI/bge-small-en-v1.5 (384-dim)

- **Why:** great trade-off between **speed**, **cost**, and **quality** for English FAQs.
- **Effect:** better matches → more confident answers → fewer escalations.
- **Notes:** smaller vectors (384) mean less storage and slightly faster searches. Easy to switch to bigger BGE models later if needed.

LLM: Groq Llama-3.1-8b-instant

- **Why:** fast responses and low cost for short answers and structured ticket summaries.
- **Effect:** keeps the app snappy (good UX) and cheap to run.
- **Notes:** the prompts are deliberately strict—answers must be **grounded in retrieved FAQs**; otherwise the bot **asks a follow-up** or **escalates**. You can upgrade to a larger Groq/OpenAI model later if you expand the scope.

Guardrails built into the prompts

- **Answering:** “Use only the provided FAQs. Be concise. Add citations like [FAQ-12]. If you can’t answer, say you’re not sure.”
- **Clarifying:** “Ask **one** short question that helps disambiguate the user’s issue.”
- **Ticket draft:** “Return **strict JSON** with **title, summary, steps, severity, environment, faq_refs.**” (We validate this JSON server-side.)

What we didn’t pick (and why)

- **Huge embeddings / huge LLMs:** more cost/latency than needed for FAQ-style Q&A.
- **Separate vector DB:** Postgres + pgvector keeps ops simple for a student budget.
- **Full fine-tuning:** not necessary for FAQs; retrieval + prompting works well and is cheaper.

9) RAG Methodology (Plain-English)

What RAG means here

Before the bot answers, it searches your FAQs to find **the most relevant passages**, then writes the reply **only** from those passages.

How it works step-by-step

1. Prepare knowledge

- Every FAQ answer gets turned into a 384-dim vector using the embedding model.
- Those vectors are stored in Postgres with **pgvector**.

2. Understand the user question

- The question is also embedded into a 384-dim vector.

3. Find likely answers

- We run a **similarity search** (top-k) in Postgres to get the best-matching FAQs.

4. Decide what to do

- If the top match is **strong** (above a similarity threshold), we answer using only those FAQs (and add citations like `[FAQ-12]`).
- If it's **weak**, we don't guess – the bot asks **one short follow-up** to clarify.

5. Escalate when needed

- If the user says “create ticket” (or confidence stays low), we summarize the **entire chat** into a clean support ticket: **title, steps, environment (web/iOS/Android/API), and FAQ refs.**

Why this avoids hallucinations

- The LLM is repeatedly told: *“Use only the provided FAQs; if unsure, say so.”*
- Similarity thresholds block “creative” answers when the match is poor.

10) End-to-End Flow (Step-by-Step)

A) Answering a question

1. **Type a question** in the chat.
2. Server embeds the question → searches pgvector → gets top FAQs.
3. If confidence is **high**:
 - LLM writes a **short, grounded answer** and adds citations ([FAQ-id]).
4. If confidence is **low**:
 - LLM returns **one targeted follow-up** (e.g., *"Are you seeing an error or a blank screen?"*).
 - The user's reply triggers another round; if still weak, suggest creating a ticket.

B) Creating a ticket

1. User clicks **Create Ticket** (or types "create ticket").
2. Server loads the **entire conversation** (not just the last few messages).
3. If it's very long, the **oldest** part is compressed; recent turns stay verbatim.
4. LLM returns a **strict JSON draft**:
 - **title** (clear, human-readable)
 - **summary** (problem + what was tried + what happened)
 - **steps** (bullet list of reproduction steps)
 - **severity** (low/high/critical)

- `environment` (web/iOS/Android/API/unknown)
- `faq_refs` (e.g., `["135","209"]`)

5. Server **inserts the ticket** and replies: *"Ticket #123 created."*

6. The ticket appears in `/tickets` with title, status, priority, created date, and the full summary.

11) UI Walkthrough (with Screenshot Placeholders)

Replace each placeholder with your own screenshot later.

1. Support page (chat)

- **Left sidebar**: list of previous conversations + **"New chat"** button.
- **Right panel**: current chat thread (user + assistant messages).
- *[Screenshot Placeholder: full support page]*

2. Answer with citations

- Bot's message ends with references like `[FAQ-135]`.
- Clicking an FAQ id (optional) can show the full answer or link to an FAQ page.
- *[Screenshot Placeholder: answer with citations]*

3. Low confidence → Follow-up

- A single, short clarifying question appears; suggestions of related FAQs may show beneath.
- *[Screenshot Placeholder: one-line follow-up + suggestions]*

4. Create ticket

- A button under the chat (or user types "create ticket").
- Confirmation toast: *"Ticket #123 created."*
- *[Screenshot Placeholder: ticket creation confirmation]*

5. Tickets list

- Shows `id`, `title`, `status`, `priority`, `created_at`.
- Clicking a ticket opens details.
- *[Screenshot Placeholder: tickets list]*

6. Ticket detail

- **Title** (clear and descriptive)
- **Summary** (what happened, steps, environment)
- **FAQ refs** (IDs included in the summary)
- *[Screenshot Placeholder: ticket detail page]*

12) Key API Endpoints (Human-Friendly)

- `POST /api/answer` → Stateless RAG answer with citations.
Body: `{ "question": "...", "sources": [...] }` • **Returns:** `{ found, answer, sources: [...] }`

- `POST /api/chat` → Stateful chat (persists if logged in), can escalate to ticket.
Body: { message, conversationId?, forceTicket?, history? }
Returns: { action: 'answer'|'clarify'|'ticket_created'|'login_required', ... }
- `GET /api/my/conversations` → List user's conversations (title + last activity).
- `GET /api/my/messages?id=...` → Messages for a conversation.
- `GET /api/my/tickets` → List user tickets.
- `POST /api/seed-faq` → Seed FAQs from CSV/HF (support `?force=true`).

13) Deployment & Configuration

Environment Variables (Vercel + Supabase):

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `SUPABASE_SERVICE_ROLE_KEY` (server-only for seeding)
- `HF_TOKEN` (Hugging Face Inference)
- `GROQ_API_KEY` (Groq LLM)

Auth URLs (Supabase → Authentication → URL Configuration):

- **Site URL:** your Vercel domain (e.g., `https://yourapp.vercel.app`)
- **Additional Redirects:** include the same Vercel URL and `http://localhost:3000` for local dev.

RLS Policies: enable on `conversations`, `messages`, `tickets` so users only see their own data.